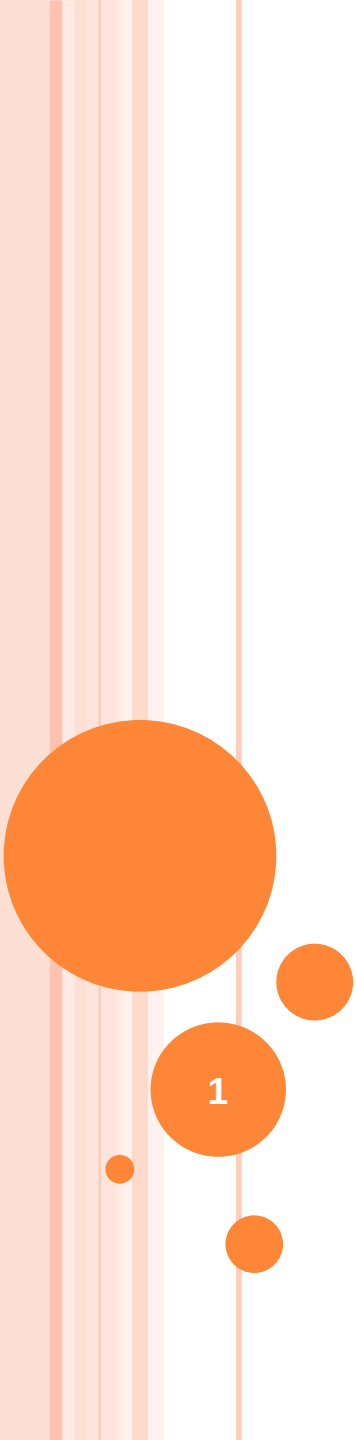




A CLOSER LOOK AT CLASSES

1

CLASS DECLARATION

```
class Dog:

    species = "canine" # Class attribute

    def __init__(self, name, age):

        """Initializes the dog's name and age."""

        self.name = name # Instance attribute

        self.age = age # Instance attribute

    def bark(self):

        """A method for the dog to bark."""

        return f'{self.name} says woof!'
```

OBJECT CREATION

Create an instance (object) of the class

```
my_dog = Dog("Buddy", 3)
```

Access attributes and call methods

```
print(f"{my_dog.name} is {my_dog.age} years old.")
```

```
print(my_dog.bark())
```

PRIVATE AND PUBLIC PROPERTY

Private" Variables

- ❑ Private variables are intended for internal use within the class and should not be accessed directly from outside.
- ❑ Python offers a mechanism called name mangling to make direct external access harder.
- ❑ Syntax: Prefix the variable name with a double underscore (`__`) (e.g., `__private_var`).
- ❑ Access: They can be accessed within the class methods but attempting direct access from outside will raise an `AttributeError`

PRIVATE, PUBLIC AND PROTECTED PROPERTY

Private" Variables

- ❑ Private variables are intended for internal use within the class and should not be accessed directly from outside.
- ❑ Python offers a mechanism called name mangling to make direct external access harder.
- ❑ Syntax: Prefix the variable name with a double underscore (`__`) (e.g., `__private_var`).
- ❑ Access: They can be accessed within the class methods but attempting direct access from outside will raise an `AttributeError`

PRIVATE, PUBLIC AND PROTECTED PROPERTY

Public Variables

- ❑ Public variables can be accessed and modified freely from anywhere, both inside and outside the class.
- ❑ Syntax: No special prefix is required

PRIVATE, PUBLIC AND PROTECTED PROPERTY

Protected Variables

- ❑ In Python, "protected" variables are defined by prefixing its name with a single underscore (`_`).
- ❑ Convention, Not Restriction: The leading underscore is a signal to other developers that the variable is intended for internal use within the class or its subclasses.
- ❑ Python's interpreter does not actually prevent you from accessing it from outside the class.
- ❑ Protected variables can be accessed from within the defining class, and also by any classes that inherit from it (subclasses).

PUBLIC, PRIVATE AND PROTECTED FUNCTIONS

- ❑ Public, private and protected functions are also declared following the same fashion of variable declaration within a class.
- ❑ Private function can't be accessed outside.
- ❑ Public function can be accessed from anywhere.
- ❑ Protected function can be accessed outside but not recommended.


```
class Person:
    def __init__(self, name, age, salary):
        # Public variable
        self.name = name

        # Protected variable (convention: single
underscore)
        self._age = age

        # Private variable (name mangling: double
underscore)
        self.__salary = salary
```

Public method

```
def show_info(self):  
    print("Public Method:")  
    print(f"Name: {self.name}")  
    print(f"Age: {self._age}")  
    print(f"Salary: {self.__salary}")
```

Protected method

```
def _update_age(self, new_age):  
    print("Protected Method:")  
    self._age = new_age
```

Private method

```
def __update_salary(self, new_salary):  
    print("Private Method:")  
    self.__salary = new_salary
```

Public method calling private method

```
def promote(self, increment):  
    self.__update_salary(self.__salary + increment)
```

```
p = Person("Faruk", 25, 50000)
```

Public access → Allowed

```
print(p.name)
```

```
p.show_info()
```

Protected access → Allowed but discouraged

```
print(p._age)
```

```
p._update_age(26)
```

Private access → Not directly accessible

print(p.__salary) ✗ AttributeError

p.__update_salary(60000) ✗ AttributeError

Name mangling (possible but not recommended)

print(p._Person__salary) # Accessing private variable

STATIC VARIABLE

- ❑ A class variable is declared by assigning a value to a variable name in the main body of the class definition
- ❑ Class Variable is one copy shared by the objects of that class.

STATIC VARIABLE

- ❑ A class variable is declared by assigning a value to a variable name in the main body of the class definition
- ❑ Class Variable is one copy shared by the objects of that class.

```
class MyClass:
```

```
    # This is a class variable (static variable)
```

```
    class_variable = "I am shared by all instances"
```

```
    count = 0 # Useful for a counter
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
        # Access and modify using the class name
```

```
        MyClass.count += 1
```



```
def display_info(self):  
    # Access using self can work, but MyClass is clearer  
    print(f'Name: {self.name}, Count: {MyClass.count}, Shared  
var: {self.class_variable}')
```

```
# Access directly using the class name  
print(f'Initial count: {MyClass.count}')
```

Create instances

obj1 = MyClass("Alice")

obj2 = MyClass("Bob")

Access via instances

obj1.display_info()

obj2.display_info()

Modify the class variable using the class name

MyClass.class_variable = "The value has changed for everyone"

obj1.display_info() *# Both objects see the updated value*

THANKS